

High-Level Design (HLD)

1. Definition

High-Level Design (HLD) describes the **overall architecture of a system**—how different components interact, how data flows, and how the system scales.

It focuses on:

- System structure (big picture)
- Component interactions
- Scalability & reliability

2. Purpose of HLD

HLD answers:

- How will the system handle **millions of users**?
- How will components **communicate**?
- How will the system **scale and fail gracefully**?

Goal:

Design a system that is **scalable, reliable, maintainable, and efficient**

3. What HLD Includes

Core Components:

- Clients (Web / Mobile)
- Servers (App servers)
- Databases
- Caches
- Load Balancers
- APIs

- Message Queues

System-Level Concerns:

- Scalability
- Availability
- Fault tolerance
- Consistency
- Latency

4. HLD vs LLD

Aspect	HLD	LLD
Focus	System architecture	Code-level design
Level	High-level	Detailed
Includes	Services, DB, APIs	Classes, methods
Used by	Architects, Senior Engineers	Developers

Example:

- HLD → “We use a load balancer + microservices”
- LLD → “This class handles payment processing”

5. Typical HLD Flow

When solving any HLD problem:

Step 1: Requirements Gathering

- Functional (features)
- Non-functional (scale, latency)

Step 2: High-Level Architecture

None

Client → Load Balancer → App Servers → Database



Cache

Step 3: Data Flow

- How request travels through system

Step 4: Scaling Strategy

- Horizontal scaling
- Caching
- DB replication/sharding

Step 5: Bottlenecks & Solutions

- DB overload → caching
- High traffic → load balancer

6. Key Concepts in HLD

♦ Scalability

- Handle increasing users

♦ Availability

- System should remain operational

♦ Consistency

- Data correctness across system

♦ Fault Tolerance

- System survives failures

7. Monolith vs Distributed Systems

Monolithic System

- Everything in one application

Distributed System

- Multiple services communicating

HLD mainly focuses on: **Distributed Systems**

8. Real-World Example

Example: Instagram Feed

HLD includes:

- Feed service
- User service
- Media storage
- CDN
- Database
- Cache (Redis)

You don't write code, you design: **how these pieces interact**

9. Why HLD is Important

Companies test HLD to evaluate:

- Problem-solving ability
- Scalability thinking
- Trade-off decisions
- Real-world engineering skills

10. Common Mistakes

- Jumping into solution without requirements
- Ignoring scalability
- Not discussing trade-offs
- Overcomplicating early design
- Not explaining data flow

12. Final Summary

- HLD = Big picture system design
- Focuses on architecture, scalability, and interactions
- Does NOT include code-level details
- Used to design large-scale distributed systems